

MIEMBROS DEL EQUIPO — GRUPO 11

Líder: Sara Palacio Zapata
Miembro: Julián Velásquez Salas

Tarea - TIA-03 (Unidad 2 - Tarea 1)

Implementación de la estructura completa de una base de datos (DDL)

- Tarea en Equipo
- Peso: 15%
- Práctica. Caso de Estudio
- Diccionario de Datos Físico - DBMS - PostgreSQL - pgAdmin4

MIEMBROS DEL EQUIPO:

- Líder
- Miembro:

Descripción de la Tarea

1. Contextualización

Hemos identificado una necesidad clara: mejorar la comunicación y la interacción entre los estudiantes. Aunque tenemos plataformas institucionales, a menudo son formales y no facilitan la conexión casual, el intercambio de conocimientos entre pares, la formación de grupos de estudio o la organización de eventos informales.

La comunidad educativa necesita una plataforma donde puedan:

- Crear perfiles para los siguientes usuarios: estudiante, docente, empleado, empresario, invitado público en general.
- Crear perfiles para diferentes actividades: compartir su área de estudio, intereses, habilidades en diferentes disciplinas o deportes, etc.
- Conectar con compañeros: encontrar y seguir a otros estudiantes con intereses similares o de cursos avanzados que puedan ofrecer mentoría.
- Publicar actualizaciones: compartir preguntas sobre tareas, recursos útiles, noticias de la industria tecnológica o incluso memes relevantes para la vida universitaria.
- Crear y unirse a grupos: organizar grupos de estudio para materias específicas, equipos para hackatones o clubes de interés (videojuegos, programación competitiva, etc.).
- Programar eventos: anunciar y coordinar reuniones de estudio, talleres, o actividades sociales.
- Comprar o vender artículos o servicios: todos los usuarios podrán realizar estas operaciones

El objetivo con este reto es diseñar la base de datos que soportará una red social con estas características. Este es un problema real que resolverán usando las herramientas y conceptos aprendidos. El objetivo de este reto es tomar el modelo lógico que ya crearon para la red social estudiantil pascualina y lo implementen como un modelo físico funcional en un Sistema de gestión de

base de datos (SGBD) real, aplicando rigurosamente las reglas y estándares del lenguaje de definición de datos (DDL).

Para lograrlo, aplicarán todo lo aprendido con el libro *“Estructura, reglas y estándares”*, que hace parte de los recursos educativos de la unidad; así como todo el material compartido por el docente y la plataforma.

Es importante la identificación correcta de los tipos de datos del SGBD en el cual implementará la base de datos. Los estudiantes deberán elaborar un Diccionario de Datos Físico, coherente y consistente al Diccionario de Datos Genérico elaborado previamente. Este diccionario físico debe documentar cada tabla y cada campo, indicando nombre, tipo de dato, si es clave primaria, clave foránea, si es obligatorio (no nulo) o único, y su descripción funcional dentro del sistema. Por lo tanto, se recomienda mucha atención en la conversión de los tipos de datos genéricos en los tipos de datos del SGBD.

2. Instrucciones de realización de la tarea

Seleccionen el SGBD que consideren más adecuado, de acuerdo con las necesidades de la red social estudiantil pascualina. Luego, utilizando una herramienta gráfica, como pgAdmin4 o MySQL Workbench, se enfocarán en la correcta implementación del modelo, prestando especial atención a la identificación de tablas, campos, los tipos y tamaños de datos, y, fundamentalmente, la definición de las claves primarias y las relaciones foráneas que dan coherencia al sistema. El proceso incluye una etapa de presentación y justificación de las decisiones de modelado, que permitirán la retroalimentación grupal guiada por el docente, para refinar el trabajo antes de la entrega final.

Para evidenciar su aprendizaje, deberán organizar su trabajo en un repositorio de Git que contendrá varios entregables clave:

- Un informe técnico que justifique sus decisiones.
- Diccionario de Datos Físico (hoja de cálculo)
- Los Scripts con el modelo físico (DDL). Nota: Creación y Modificación de Tablas
- Un video corto donde sustenten el trabajo realizado y las conclusiones individuales que reflejen su aprendizaje personal.

Esta tarea les permitirá desarrollar competencias esenciales como el reconocimiento de sentencias DDL y la implementación de una base de datos en un SGBD, habilidades fundamentales para cualquier desarrollador de software.

Tengan en cuenta la participación activa en el “Foro. Trabajando en la implementación de la Red Social Estudiantil Pascualina”, a través del cual se evidenciará el trabajo colaborativo y continuo a lo largo del desarrollo de la tarea.

3. Criterios de entrega y valoración de la tarea

Los entregables finales para esta tarea, serán almacenados bajo la siguiente arquitectura:

Carpeta raíz: Tarea3. En esta carpeta estarán dos carpetas con los entregables: Informe y Video.

Carpetas	Contenido requerido	Observaciones
/tarea3	Todas las subcarpetas y archivos de la tarea.	No aplica
tarea3/informe	Informe detallado	.pdf
tarea3/Resultados	Diccionario de Datos Físico (hoja de cálculo)	.xlsx
	Script de creación y Script de modificación de la base de datos (archivos planos con instrucciones DDL)	.sql
tarea3/video	Video corto (5-10 minutos) con todos los miembros explicando una parte del trabajo y ejecutando instrucciones en el SGBD	Enlace a YouTube

Evidencias de aprendizaje evaluadas

Evidencia	Habilidad evaluada
Informe con el paso a paso de la propuesta de solución, teniendo en cuenta: • Identificación del SGBD y justificación de su elección (contemplen las ventajas y desventajas en comparación con otras opciones consideradas). • Descripción del modelo físico en el que expliquen cómo el modelo lógico fue transformado. • Descripción de cada tabla implementada con: nombre de la tabla, listado de campos (atributos) con su tipo de dato y tamaño (si aplica), justificación de la elección de tipos de datos para campos clave (por ejemplo, por qué eligieron VARCHAR en lugar de TEXT para un campo específico, o INT en lugar de BIGINT), identificación de la clave primaria (PK) y justificación de su elección. • Descripción de cómo se implementaron las relaciones entre tablas. Para cada relación incluir: tablas involucradas (tabla padre, tabla hija y/o tabla pivote), campos que conforman la clave foránea, especificación de las acciones referenciales (ON DELETE, ON UPDATE, ON CASCADE) y su justificación. • El informe debe referenciar el script DDL y explicar su estructura. • Conclusiones individuales sobre el proceso de solución, destacando dificultades, dudas, aspectos relevantes del proceso.	Capacidad de análisis y redacción técnica Reflexión crítica y aprendizaje personal.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Grupo 11

Evidencia	Habilidad evaluada
Modelo físico con: ● Inventario completo de tablas de la base de datos. ● Tablas con nombres según las reglas y la nomenclatura. ● Campos correctamente identificados con tamaño y tipo asociado. ● Relaciones adecuadas al caso (claves primarias y foráneas). ● Uso correcto las reglas, nomenclatura y estándares para BD físicas.	Comprensión del modelo físico
Video de sustentación	Comunicación oral y trabajo en equipo
Específicos	Competencia en gestión de versiones y organización digital.

Analicen los criterios de evaluación y verifiquen su cumplimiento..

Informe con resultados

1.- Diagrama Entidad-Relación de Chen (corregido)

El siguiente pantallazo muestra el Diagrama ER corregido, elaborado en la pestaña correspondiente de la hoja de cálculo de resultados. El diagrama refleja las 30 entidades del modelo lógico, sus atributos, relaciones y cardinalidades.

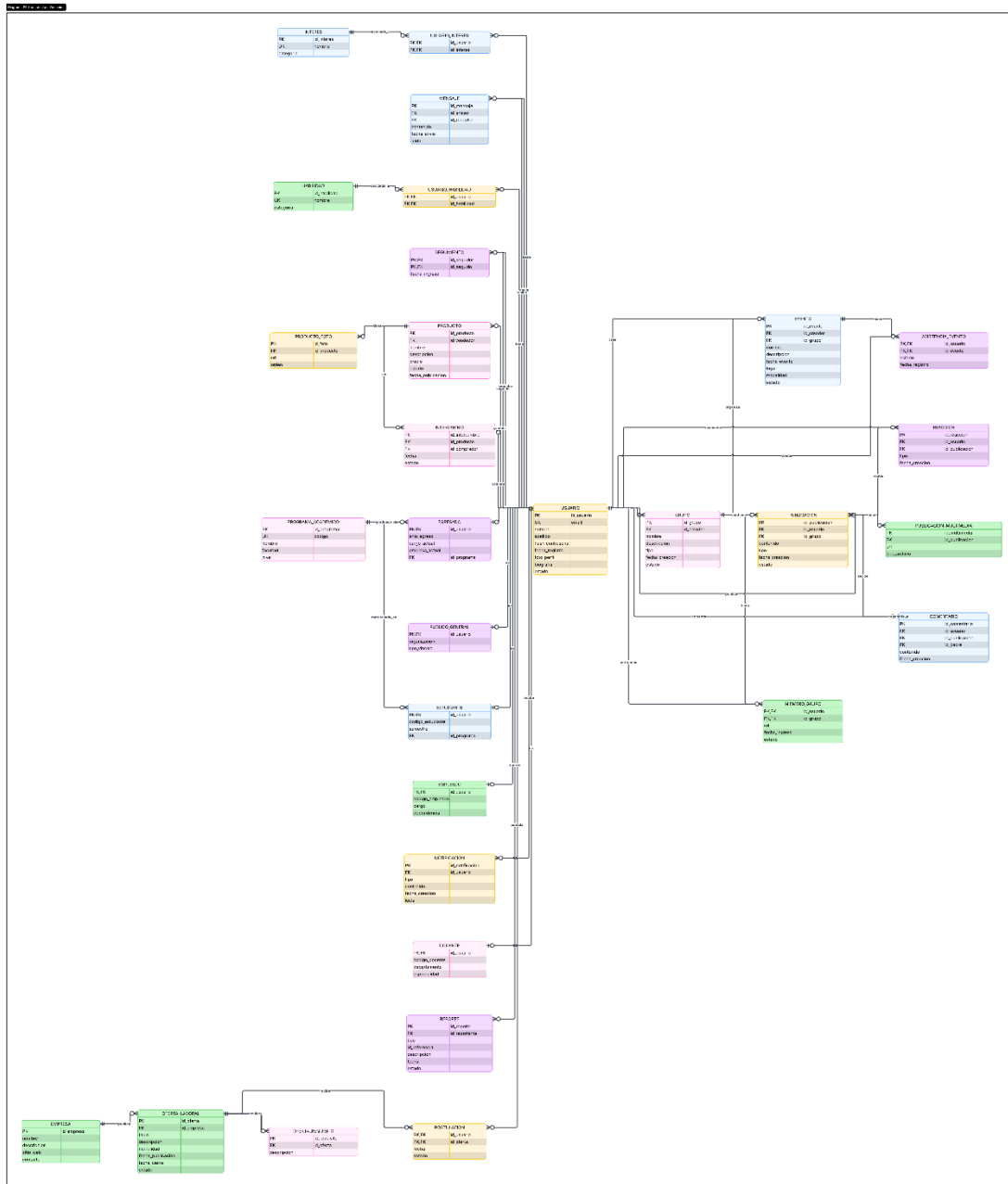


Ilustración 1 Diagrama ER, realizado en LucidChard

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Grupo 11

2.- Inventario de Tablas del Diccionario de Datos Genérico (corregido)

A continuación, se presenta el inventario corregido de las 30 tablas resultantes del modelo lógico. Cada tabla está nombrada en snake_case, respetando la nomenclatura estándar para bases de datos físicas. El inventario también registra las observaciones relevantes sobre claves primarias, foráneas y restricciones de unicidad.

Tablas resultantes del Diccionario Genérico (Modelo Lógico)			
#	Tabla	Descripción Tabla	Observaciones
1	programa_academico	Catálogo de programas académicos ofrecidos por la institución	PK: id_programa UK: codigo
2	habilidad	Catálogo de habilidades que los usuarios pueden asociar a su perfil	PK: id_habilidad UK: nombre
3	interes	Catálogo de áreas de interés disponibles para los usuarios	PK: id_interes UK: nombre
4	empresa	Empresas registradas en la plataforma para publicar ofertas laborales	PK: id_empresa
5	usuario	Supertipo que agrupa a todos los tipos de usuarios de la red social	PK: id_usuario UK: email
6	estudiante	Subtipo de usuario: estudiante activo de la institución	PK: id_usuario FK: id_usuario, id_programa
7	docente	Subtipo de usuario: docente vinculado a la institución	PK: id_usuario FK: id_usuario
8	empleado	Subtipo de usuario: empleado administrativo de la institución	PK: id_usuario FK: id_usuario
9	egresado	Subtipo de usuario: graduado de la institución	PK: id_usuario FK: id_usuario, id_programa
10	publico_general	Subtipo de usuario: persona externa sin vínculo institucional formal	PK: id_usuario FK: id_usuario
11	usuario_habilidad	Tabla pivote N:M entre usuario y habilidad	PK compuesta: id_usuario + id_habilidad
12	usuario_interes	Tabla pivote N:M entre usuario y área de interés	PK compuesta: id_usuario + id_interes
13	seguimiento	Relación reflexiva N:M que modela el seguimiento entre usuarios	PK compuesta: id_seguidor + id_seguido
14	grupo	Grupos creados por usuarios para organizar actividades colectivas	PK: id_grupo FK: id_creador
15	miembro_grupo	Tabla pivote N:M entre usuario y grupo con rol del miembro	PK compuesta: id_usuario + id_grupo
16	evento	Eventos académicos o sociales creados dentro de la plataforma	PK: id_evento FK: id_creador, id_grupo
17	asistencia_evento	Registro de asistencia o confirmación de usuarios a eventos	PK compuesta: id_usuario + id_evento
18	publicacion	Publicaciones realizadas por usuarios en el feed de la red social	PK: id_publicacion FK: id_usuario, id_grupo
19	publicacion_multimedia	Archivos multimedia adjuntos a una publicación	PK: id_multimedia FK: id_publicacion
20	comentario	Comentarios realizados por usuarios sobre publicaciones	PK: id_comentario FK: id_usuario, id_publicacion, id_padre
21	reaccion	Reacciones de usuarios a publicaciones (like, love, etc.)	PK: id_reaccion UK: (id_usuario, id_publicacion)
22	producto	Productos o servicios publicados en el marketplace de la plataforma	PK: id_producto FK: id_vendedor
23	producto_foto	Fotografías asociadas a un producto del marketplace	PK: id_foto FK: id_producto
24	intercambio	Registro de transacciones de compra o intercambio de	PK: id_intercambio FK: id_producto,

Ilustración 2 Extracto de Diccionario genérico

El inventario incluye 30 tablas distribuidas en los siguientes grupos funcionales:

- Supertipo y subtipos de usuario: usuario, estudiante, docente, empleado, egresado, publico_general.
- Catálogos: programa_academico, habilidad, interes, empresa.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Grupo 11

- Relaciones N:M y reflexivas: usuario_habilidad, usuario_interes, seguimiento, miembro_grupo, asistencia_evento, postulacion.
- Contenido social: publicacion, publicacion_multimedia, comentario, reaccion, grupo, evento.
- Marketplace (Cambalache Store): producto, producto_foto, intercambio.
- Bolsa de trabajo: oferta_laboral, oferta_requisito.
- Comunicación y gestión: mensaje, notificacion, reporte.

3.- Tabla Comparativa de tipos de Datos

Tabla Comparativa de tipos de Datos La siguiente tabla presenta la equivalencia entre los tipos de datos genéricos utilizados en el Diccionario de Datos Genérico y los tipos de datos nativos de PostgreSQL aplicados en el modelo físico.

Tipos de Datos - Tabla Comparativa		
#	Tipo de Dato Genérico	Tipos de Dato - PosgreSQL
1	AUTOINCREMENTABLE	SERIAL
2	UUID	UUID
3	ENTERO	INT / SMALLINT
4	DECIMAL	NUMERIC(p,s)
5	LÓGICO	BOOLEAN
6	TEXTO	TEXT
7	CARACTER	VARCHAR(n)
8	FECHA	DATE
9	FECHA_HORA	TIMESTAMP
10	IMAGEN	VARCHAR(255) — URL externa
11	AUDIO	VARCHAR(255) — URL externa
12	URL	VARCHAR(255)
13	ENUM	VARCHAR(n) con CHECK
14	TEXTO_CORTO (opcional)	CHAR(n)
15	BINARIO (opcional)	BYTEA
16	JSON	JSONB

Ilustración 3 Tabla comparativa de Datos

Las decisiones más relevantes en la conversión de tipos son:

- AUTOINCREMENTABLE → SERIAL: PostgreSQL implementa los secuenciales automáticos con SERIAL, que internamente crea una secuencia y la asocia al campo. Se prefirió sobre BIGSERIAL dado que el volumen esperado de registros no supera los límites de un entero de 32 bits.
- CARACTER → VARCHAR(n): Se usó VARCHAR con longitud explícita en lugar de TEXT para campos acotados (nombres, emails, códigos), porque permite establecer restricciones de longitud en el motor y comunica mejor la intención del diseño al equipo.
- TEXTO → TEXT: Para campos de contenido libre como biografías, descripciones y cuerpos de publicaciones, se eligió TEXT dado que su longitud es impredecible y no tiene sentido limitarla arbitrariamente.
- ENUM → VARCHAR(n) con CHECK: PostgreSQL soporta tipos ENUM nativos, pero se optó por VARCHAR con restricción CHECK para facilitar futuras extensiones de los valores válidos sin necesidad de alterar el tipo de dato.
- LÓGICO → BOOLEAN: Mapeo directo. Se usa para los campos leído y leída de mensaje y notificación.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Grupo 11

- FECHA → DATE y FECHA_HORA → TIMESTAMP: Se diferencian según la necesidad de precisión horaria. Las fechas de cierre de ofertas laborales usan DATE; los registros de eventos y transacciones usan TIMESTAMP.
- DECIMAL → NUMERIC(10,2): Para el campo precio de producto, se usa NUMERIC con precisión y escala explícitas para evitar errores de redondeo propios de los tipos de punto flotante.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Grupo 11

4.- Diccionario de Datos Físico (SGBD PostgreSQL)

El Diccionario de Datos Físico se elaboró en la hoja de cálculo de resultados (archivo resultados.xlsx), en la pestaña 'DICCIONARIO DATOS (Físico)'. Los nombres y descripciones de cada tabla se referencian desde la pestaña de inventario, garantizando consistencia entre las secciones. A continuación, se presenta un pantallazo de la sección inicial del diccionario.

DICCIONARIO DE DATOS FÍSICO — Red Social Estudiantil Pascualina		
INVENTARIO DE TABLAS		
#	Tabla	Descripción Tabla
1	programa_academico	Catálogo de programas académicos ofrecidos por la institución
2	habilidad	Catálogo de habilidades que los usuarios pueden asociar a su perfil
3	interes	Catálogo de áreas de interés disponibles para los usuarios
4	empresa	Empresas registradas en la plataforma para publicar ofertas laborales
5	usuario	Supertipo que agrupa a todos los tipos de usuarios de la red social
6	estudiante	Subtipo de usuario: estudiante activo de la institución
7	docente	Subtipo de usuario: docente vinculado a la institución
8	empleado	Subtipo de usuario: empleado administrativo de la institución
9	egresado	Subtipo de usuario: graduado de la institución
10	publico_general	Subtipo de usuario: persona externa sin vínculo institucional formal
11	usuario_habilidad	Tabla pivote N:M entre usuario y habilidad
12	usuario_interes	Tabla pivote N:M entre usuario y área de interés
13	seguimiento	Relación reflexiva N:M que modela el seguimiento entre usuarios
14	grupo	Grupos creados por usuarios para organizar actividades colectivas
15	miembro_grupo	Tabla pivote N:M entre usuario y grupo con rol del miembro
16	evento	Eventos académicos o sociales creados dentro de la plataforma
17	asistencia_evento	Registro de asistencia o confirmación de usuarios a eventos
18	publicacion	Publicaciones realizadas por usuarios en el feed de la red social
19	publicacion_multimedia	Archivos multimedia adjuntos a una publicación
20	comentario	Comentarios realizados por usuarios sobre publicaciones
21	reaccion	Reacciones de usuarios a publicaciones (like, love, etc.)
22	producto	Productos o servicios publicados en el marketplace de la plataforma
23	producto_foto	Fotografías asociadas a un producto del marketplace
24	intercambio	Registro de transacciones de compra o intercambio de productos
25	oferta_laboral	Ofertas de trabajo publicadas por empresas registradas
26	oferta_requisito	Requisitos específicos asociados a una oferta laboral
27	postulacion	Postulaciones de usuarios a ofertas laborales
28	mensaje	Mensajes directos intercambiados entre usuarios de la plataforma
29	notificacion	Notificaciones generadas por el sistema para informar eventos al usuario
30	reporte	Reportes de contenido inapropiado realizados por usuarios

Ilustración 4 Diccionario de datos

4.1 Selección del SGBD y justificación

Para la implementación del modelo físico de la Red Social Estudiantil Pascualina se seleccionó PostgreSQL como Sistema Gestor de Base de Datos (SGBD). La decisión se tomó comparando tres opciones:

SGBD	Licencia	Escalabilidad	Adecuación al proyecto
PostgreSQL ✓ (elegido)	Código abierto / gratuita	Alta — soporta JSON, índices avanzados, extensiones	Muy alta — cumple todos los requisitos del proyecto
MySQL	Código abierto / dual	Alta — amplio soporte de réplicas	Alta — popular, pero sin soporte nativo de herencia de tablas
SQLite	Dominio público	Baja — diseñado para apps embebidas	Baja — no apto para múltiples usuarios concurrentes

PostgreSQL fue seleccionado por las siguientes razones concretas:

- Soporta el tipo SERIAL para autoincrementales y NUMERIC para valores monetarios con precisión exacta.
- Permite restricciones CHECK complejas, fundamentales para validar estados (activo/inactivo/suspendido), tipos de reacción y rangos de semestre.
- Su soporte de claves foráneas con acciones referenciales (CASCADE, SET NULL, RESTRICT) cubre todos los patrones de relación del modelo.
- pgAdmin4 ofrece una interfaz gráfica completa para ejecutar y validar los scripts DDL.
- Es el SGBD recomendado por el docente para la asignatura.

4.2 Descripción del modelo físico

El modelo lógico fue transformado al modelo físico aplicando las siguientes reglas:

- Cada entidad del modelo ER se convirtió en una tabla con sus atributos como columnas, tipos de datos nativos de PostgreSQL y restricciones explícitas (NOT NULL, UNIQUE, CHECK).
- La herencia de USUARIO se implementó mediante el patrón tabla por subtipo: una tabla padre usuario almacena los atributos comunes, y cada subtipo (estudiante, docente, empleado, egresado, publico_general) tiene su propia tabla con la PK que también actúa como FK hacia usuario.
- Las relaciones N:M se materializaron en tablas pivote: usuario_habilidad, usuario_interes, miembro_grupo, asistencia_evento y postulacion.
- La relación reflexiva de seguimiento entre usuarios se implementó en la tabla seguimiento con dos FKs hacia usuario (id_seguidor e id_seguido) y una restricción CHECK que impide el auto-seguimiento.
- Las tablas de 1FN (publicacion_multimedia, producto_foto, oferta_requisito) separan atributos multivaluados en tablas dependientes con FK y CASCADE.
- Todo el modelo se encapsula en el esquema pascualina mediante CREATE SCHEMA y SET search_path, siguiendo el estándar de organización por esquema.

4.3 Descripción de tablas implementadas

A continuación, se describen las decisiones de diseño más relevantes por grupo de tablas:

Supertipo y subtipos de usuario

La tabla usuario almacena id_usuario (SERIAL, PK), email (VARCHAR(150), NOT NULL, UNIQUE), nombre y apellido (VARCHAR(100)), contrasena (VARCHAR(255) — almacena el hash, nunca texto plano), fecha_registro (TIMESTAMP, DEFAULT NOW()), foto_perfil (VARCHAR(255)), biografia (TEXT) y estado (VARCHAR(20) con CHECK en 'activo', 'inactivo', 'suspendido'). Se eligió VARCHAR sobre TEXT para email porque tiene longitud máxima conocida (RFC 5321 limita a 254 caracteres) y permite optimización de índices. El campo estado usa VARCHAR con CHECK en lugar de un tipo ENUM nativo para facilitar futuras extensiones sin DDL adicional.

Los cinco subtipos comparten el mismo id_usuario como PK y FK hacia usuario con ON DELETE CASCADE — si se elimina un usuario, su registro en el subtipo desaparece también, manteniendo la integridad referencial sin huérfanos.

Catálogos

Las tablas programa_academico, habilidad e interes funcionan como catálogos independientes. Sus PKs son SERIAL y tienen restricción UNIQUE en el campo nombre para evitar duplicados semánticos. empresa también es independiente y no forma parte de la jerarquía de usuarios, pues representa organizaciones externas aliadas.

Contenido social

publicacion referencia a usuario (FK id_usuario, CASCADE) y opcionalmente a grupo (FK id_grupo, SET NULL — la publicación puede existir en el feed global sin pertenecer a un grupo). comentario implementa una FK recursiva id_padre que apunta a la misma tabla comentario, permitiendo hilos anidados; cuando se elimina un comentario padre, sus respuestas se eliminan en cascada. reaccion tiene una restricción UNIQUE sobre (id_usuario, id_publicacion) para garantizar máximo una reacción por usuario por publicación.

Marketplace

producto usa id_vendedor con ON DELETE RESTRICT — no se puede eliminar un usuario que tiene productos activos, protegiendo la integridad del historial comercial. producto_foto e intercambio dependen del producto; las fotos se eliminan en cascada, pero el intercambio usa RESTRICT para conservar el registro de transacciones.

Bolsa de trabajo

oferta_laboral depende de empresa con RESTRICT — no se puede eliminar una empresa con ofertas activas. oferta_requisito es una tabla 1FN que almacena individualmente cada requisito de la oferta, con CASCADE hacia oferta_laboral.

5.- Lenguaje de Definición de Dato (DDL) - Scripts de **CREACIÓN** de Bases de Datos

El script de creación se encuentra en el archivo scripts-crea.txt. Contiene la creación del esquema pascualina y las 30 tablas en orden estricto: primero las tablas independientes (sin claves foráneas) y luego las dependientes, garantizando que el script completo se ejecute sin errores en pgAdmin4.

La estructura del script sigue el siguiente orden de creación:

- Creación del esquema: CREATE SCHEMA IF NOT EXISTS pascualina; SET search_path TO pascualina;
- Tablas independientes: programa_academico, habilidad, interes, empresa, usuario.
- Subtipos de usuario (dependen de usuario y programa_academico): estudiante, docente, empleado, egresado, publico_general.
- Tablas pivote de catálogos: usuario_habilidad, usuario_interes.
- Relación reflexiva: seguimiento.
- Grupos y membresías: grupo, miembro_grupo.
- Eventos y asistencia: evento, asistencia_evento.
- Contenido social: publicacion, publicacion_multimedia, comentario, reaccion.
- Marketplace: producto, producto_foto, intercambio.
- Bolsa de trabajo: oferta_laboral, oferta_requisito, postulacion.
- Comunicación y gestión: mensaje, notificacion, reporte.

Cada instrucción CREATE TABLE incluye: nombre de la tabla en snake_case, definición de columnas con tipo y tamaño PostgreSQL, restricción NOT NULL donde aplica, restricciones de clave primaria (CONSTRAINT pk_*), claves foráneas (CONSTRAINT fk_*) con acciones referenciales ON DELETE y ON UPDATE, restricciones de unicidad (CONSTRAINT uq_*) y restricciones de validación (CONSTRAINT ck_*).

Grupo 11



Ilustración 5 Captura de pantalla de creación de tablas en pgAdmin

6.- Lenguaje de Definición de Dato (DDL) - Scripts de MODIFICACIÓN de Bases de Datos

El script de modificación se encuentra en el archivo scripts-modifica.txt. Contiene instrucciones ALTER TABLE, CREATE INDEX, RENAME TABLE y TRUNCATE TABLE ejecutadas en secuencia sobre el esquema pascualina. El script demuestra el manejo completo del ciclo de vida de una tabla nueva dentro del sistema.

El script se organiza en las siguientes secciones:

- 1.1 — ADD COLUMN: Se agrega el campo telefono (VARCHAR(20), NULL) a la tabla usuario para registrar un número de contacto opcional.
- 1.2 — ALTER COLUMN: Se amplía el campo nombre de VARCHAR(100) a VARCHAR(150) para soportar nombres compuestos más largos.
- 1.3.1 — CREATE TABLE: Se crea la tabla insignia, que representa logros o badges otorgados por participación activa en la plataforma.
- 1.3.2 — ADD CONSTRAINT + ADD COLUMN: Se define la PK sobre id_insignia y se agregan los campos categoria (VARCHAR(80)), puntos (INT, DEFAULT 0) y rareza (VARCHAR(20)).
- 1.3.3 — DROP COLUMN: Se elimina el campo icono_url, unificando los recursos visuales al sistema de multimedia centralizado.
- 1.3.4 — RENAME TABLE: Se renombra insignia a logro para reflejar mejor el concepto de logro alcanzado.
- 1.3.5 — ADD UNIQUE: Se agrega restricción UNIQUE al campo nombre de logro.
- 1.3.6 — ADD COLUMN + CHECK: Se agregan fecha_inicio y fecha_fin con restricción de orden entre fechas.
- 1.3.7 — ADD COLUMN + CHECK: Se agrega total_otorgados (INT, DEFAULT 0) con restricción CHECK ≥ 0 .
- 1.3.8 — ALTER COLUMN: Se amplía categoria de VARCHAR(80) a VARCHAR(120).
- 1.3.7 bis — ADD CHECK: Se agrega restricción de rango 0–1000 al campo puntos.
- 1.3.8 bis — CREATE INDEX: Se crea índice sobre el campo categoria para optimizar consultas por tipo.
- 1.3.9 — DROP COLUMN: Se elimina fecha_fin; los logros no caducan una vez habilitados.
- 1.3.10 — TRUNCATE TABLE: Se limpian todos los datos de la tabla logro con RESTART IDENTITY CASCADE.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Grupo 11

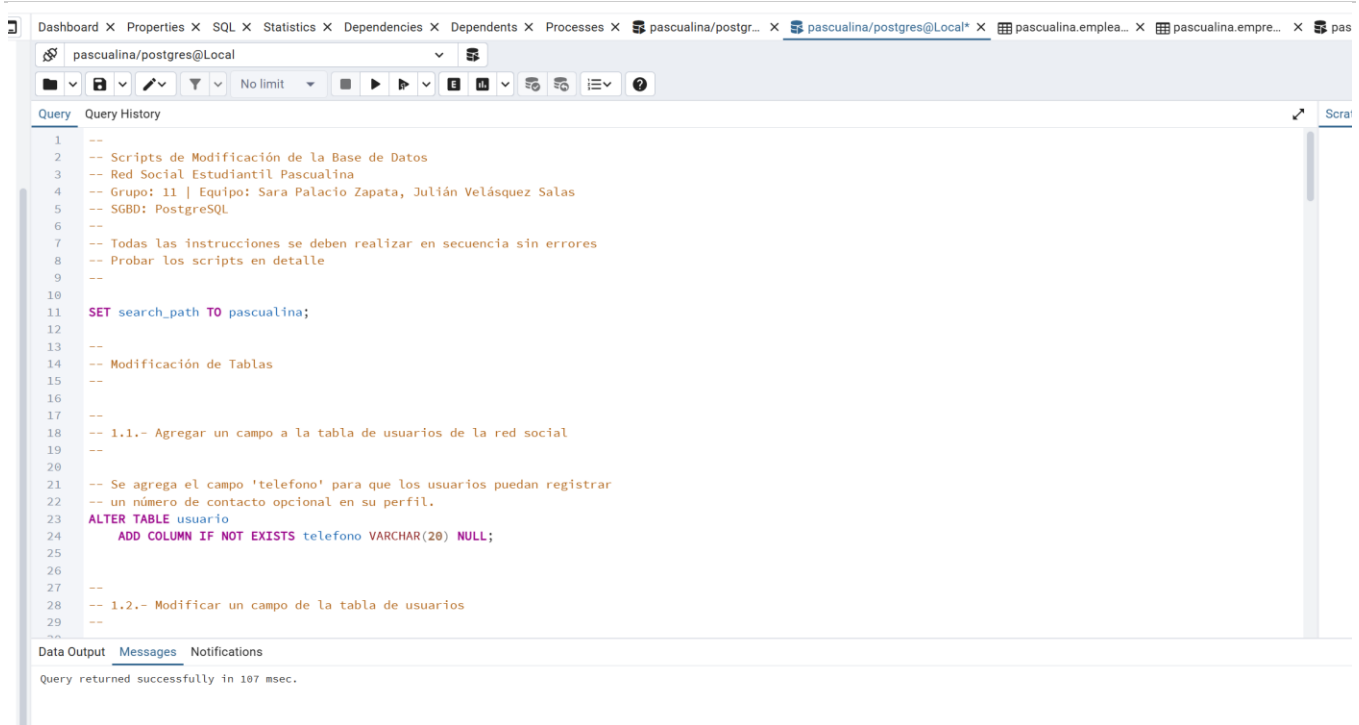


Ilustración 6 Captura de pantalla de modificación de base de datos en pgAdmin

7.- Análisis de resultados

7.1 Relaciones entre tablas e integridad referencial

La siguiente tabla resume las principales relaciones implementadas en el modelo físico, incluyendo las acciones referenciales elegidas y su justificación:

Tabla Padre	Tabla Hija / Pivote	Clave Foránea	ON DELETE / UPDATE	Justificación
usuario	estudiante	id_usuario	CASCADE / CASCADE	El subtipo no existe sin el supertipo.
usuario	docente	id_usuario	CASCADE / CASCADE	Ídem subtipo.
usuario	empleado	id_usuario	CASCADE / CASCADE	Ídem subtipo.
usuario	egresado	id_usuario	CASCADE / CASCADE	Ídem subtipo.
usuario	publico_general	id_usuario	CASCADE / CASCADE	Ídem subtipo.
programa_academico	estudiante	id_programa	SET NULL / CASCADE	El programa puede eliminarse sin perder al estudiante.
programa_academico	egresado	id_programa	SET NULL / CASCADE	El programa puede eliminarse sin perder al egresado.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Grupo 11

usuario	seguimiento	id_seguidor / id_seguido	CASCADE / CASCADE	Si se elimina un usuario, sus relaciones de seguimiento desaparecen.
usuario	publicacion	id_usuario	CASCADE / CASCADE	Las publicaciones pertenecen al usuario; se eliminan con él.
publicacion	comentario	id_publicacion	CASCADE / CASCADE	Los comentarios no tienen sentido sin la publicación.
comentario	comentario	id_padre	CASCADE / CASCADE	Hilo recursivo; al eliminar un comentario padre se eliminan sus respuestas.
publicacion	reaccion	id_publicacion	CASCADE / CASCADE	Las reacciones dependen de la publicación.
usuario	grupo	id_creador	RESTRICT / CASCADE	No se puede eliminar un usuario que creó grupos activos.
grupo	miembro_grupo	id_grupo	CASCADE / CASCADE	Tabla pivote N:M; se elimina junto con el grupo.
usuario	evento	id_creador	RESTRICT / CASCADE	No se puede eliminar al creador mientras exista el evento.
grupo	evento	id_grupo	SET NULL / CASCADE	El evento puede existir sin grupo organizador.
empresa	oferta_laboral	id_empresa	RESTRICT / CASCADE	Una oferta laboral requiere empresa; no se elimina en cascada.
oferta_laboral	oferta_requisito	id_oferta	CASCADE / CASCADE	Tabla 1FN; los requisitos no existen sin la oferta.
usuario	postulacion	id_usuario	CASCADE / CASCADE	Si el usuario se elimina, su historial de postulaciones desaparece.
usuario	producto	id_vendedor	RESTRICT / CASCADE	No se puede eliminar un usuario con productos activos en el marketplace.
producto	producto_foto	id_producto	CASCADE / CASCADE	Tabla 1FN; las fotos no existen sin el producto.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Grupo 11

producto	intercambio	id_producto	RESTRICT / CASCADE	El registro de transacción debe conservarse.
----------	-------------	-------------	-----------------------	--

7.2 Análisis general del modelo

La implementación del modelo físico de la Red Social Estudiantil Pascualina en PostgreSQL representó el paso de un diseño conceptual y lógico a una estructura ejecutable y validada en un SGBD real. Este proceso exigió tomar decisiones concretas sobre tipos de datos, tamaños, restricciones y acciones referenciales que no estaban completamente especificadas en el modelo lógico.

La consistencia entre el modelo conceptual (Diagrama ER de Chen), el modelo lógico (Diccionario de Datos Genérico) y el modelo físico (Diccionario de Datos Físico + scripts DDL) fue el criterio rector de todo el trabajo. Cada tabla del modelo físico tiene correspondencia directa con una entidad del modelo lógico, y cada campo tiene su equivalente genérico documentado en la tabla comparativa de tipos de datos.

Uno de los aspectos más relevantes fue la implementación del patrón supertipo-subtipo para los usuarios. En lugar de una única tabla monolítica con muchos campos opcionales, se optó por una tabla padre usuario con los atributos comunes y cinco tablas hijas para los subtipos específicos. Esto garantiza que cada fila de un subtipo tiene exactamente los atributos que le corresponden, evita valores NULL masivos y facilita la extensión futura del modelo.

Las tablas de normalización 1FN (publicacion_multimedia, producto_foto, oferta_requisito) demuestran la aplicación correcta del proceso de normalización: atributos multivaluados que en el modelo conceptual aparecían como listas se convirtieron en tablas dependientes con FK y eliminación en cascada.

La elección de acciones referenciales no fue arbitraria. Se aplicó CASCADE cuando la existencia del registro hijo depende completamente del padre, SET NULL cuando la relación es opcional y RESTRICT cuando se debe proteger la integridad del dato padre antes de permitir su eliminación. Esta lógica se aplicó consistentemente en las 21 relaciones foráneas del modelo.

Desde el punto de vista de los tipos de datos, la conversión de genéricos a PostgreSQL requirió decisiones justificadas caso a caso. La elección de VARCHAR con CHECK sobre ENUM nativo, de NUMERIC sobre FLOAT para precios, y de TEXT sobre VARCHAR ilimitado para contenidos libres, responden a criterios de mantenibilidad, rendimiento y semántica del dato.

En conjunto, el modelo físico resultante es coherente, ejecutable sin errores en PostgreSQL, y sienta las bases técnicas adecuadas para el desarrollo posterior de la aplicación Pascualina.

8.- Reflexiones y conclusiones individuales

Sara

La realización de esta tarea me permitió comprender de manera práctica la distancia real que existe entre un modelo lógico y su implementación física en un sistema gestor de base de datos. Durante el proceso de elaboración del Diccionario de Datos Físico, me encontré frecuentemente tomando decisiones que el modelo lógico dejaba abiertas: ¿cuántos caracteres debe tener un campo de nombre? ¿Se debe usar ENUM o VARCHAR con CHECK? ¿Cuándo aplicar CASCADE y cuándo RESTRICT? Estas preguntas, que parecen detalles técnicos menores, tienen en realidad un impacto directo en la integridad, el rendimiento y la mantenibilidad del sistema.

Una de las mayores dificultades que enfrenté fue entender el orden de creación de las tablas en el script DDL. Al principio no dimensionaba que una clave foránea obliga a que la tabla referenciada exista previamente, lo que impone un orden estricto de ejecución. Organizar las 30 tablas en el orden correcto — independientes primero, dependientes después — me hizo razonar sobre la estructura de dependencias del modelo de una forma que los diagramas no logran transmitir igual de claramente.

El trabajo con pgAdmin4 también fue una experiencia de aprendizaje importante. Ejecutar el script y ver cómo las tablas aparecen en el árbol de objetos, cómo se pueden inspeccionar las restricciones y cómo el motor rechaza instrucciones con errores de integridad referencial, concreta lo que en el papel es abstracto. Cometer errores y corregirlos en el entorno real es una forma de aprendizaje que ningún ejercicio teórico puede reemplazar.

A nivel de equipo, trabajar con Julián implicó coordinación sobre las decisiones de diseño — especialmente en los campos del script de modificación — y una revisión mutua del trabajo que mejoró la calidad final. La experiencia refuerza para mí que el diseño de bases de datos no es un ejercicio individual: las decisiones de uno afectan el trabajo de los demás y, eventualmente, a todos los usuarios del sistema.

En términos del impacto profesional, este tipo de competencias — diseñar, documentar e implementar un modelo de datos completo — son fundamentales para cualquier rol de desarrollo de software o análisis de datos. La Red Social Pascualina, aunque sea un caso de estudio, replica fielmente la complejidad de un sistema real: usuarios con roles diferenciados, contenido generado por los usuarios, transacciones, notificaciones y moderación. Haber construido su base de datos desde el diagrama ER hasta el script ejecutable me da una base sólida para proyectos futuros de mayor escala.

Julián

Esta tarea representó para mí el primer acercamiento real a la implementación de una base de datos en un entorno productivo. Pasar del diagrama ER al modelo físico ejecutable en PostgreSQL no es un proceso lineal: requiere revisar constantemente las decisiones previas, corregir inconsistencias y justificar cada elección técnica. Esa iteración constante entre el modelo conceptual, el lógico y el físico fue el aprendizaje más valioso del proceso.

Uno de los aspectos que más me costó interiorizar fue la distinción entre tipos de datos similares: cuándo usar VARCHAR versus TEXT, INT versus SMALLINT, DATE versus TIMESTAMP. No se trata solo de que el dato quede almacenado, sino de que el tipo comunique correctamente la naturaleza del dato y permita al motor optimizar su manejo. Entender que VARCHAR(20) para un código estudiantil no es arbitrario, sino que refleja un límite real del dominio del dato, cambia completamente la perspectiva con la que se diseña una tabla.

El script de modificación fue la sección que más me hizo pensar. El enunciado pedía una secuencia de operaciones sobre una tabla nueva: crearla, agregar campos, renombrarla, eliminar columnas, agregar índices y finalmente truncarla. Seguir ese ciclo completo en un solo script, garantizando que cada instrucción ejecute sin errores sobre el resultado de la anterior, requiere planificar cuidadosamente y entender el estado de la tabla en cada paso. Fue como resolver un problema de lógica en SQL.

En cuanto al trabajo en equipo, la experiencia con Sara fue fluida. Dividimos responsabilidades según las fortalezas de cada uno y mantuvimos comunicación constante para asegurar que nuestras partes del trabajo fueran coherentes entre sí. Esa dinámica de revisión mutua es exactamente lo que ocurre en un equipo de desarrollo real, y practicarla en el contexto académico es una preparación válida para el entorno profesional. Me llevo de esta tarea no solo conocimientos técnicos de SQL y PostgreSQL, sino también una metodología de trabajo colaborativo en proyectos de ingeniería de datos.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Grupo 11

Enlace a repositorio: <https://github.com/sarapalac10/bd1-redsocial-grupo11.git>

**bd1-redsocial-grupo11** Private

Watch 0

main 1 Branch 0 Tags

Add file

<> Code

**sarapalac10** Agregar documentos de tia 03 unidad 2 4a1aa8f · 34 minutes ago 🕒 14 Commits

Tarea-01	Add project report PDF and YouTube video link for Red Social...	last month
Tarea-02	Agregar documentos de tia 03 unidad 2	34 minutes ago
Tarea-03	Agregar documentos de tia 03 unidad 2	34 minutes ago
Tarea-04	Añadir carpetas Tarea-02 a Tarea-06 con Informe, Resultados ...	last month
Tarea-05	Añadir carpetas Tarea-02 a Tarea-06 con Informe, Resultados ...	last month
Tarea-06	Añadir carpetas Tarea-02 a Tarea-06 con Informe, Resultados ...	last month
README.md	Update README.md	yesterday

README

Institución Universitaria Pascual Bravo

Programa: Tecnología en Desarrollo de Software
Curso: Base de Datos I (G100)

Profesor: Jaime E. Soto U.

Grupo: #11

Propósito

Este repositorio contiene las tareas prácticas del curso **Base de Datos I**, desarrolladas como parte de la evaluación del semestre.

Cada carpeta corresponde a una de las tareas solicitadas en clase.

Estudiante

- Sara Palacio Zapata
- Julián Velásquez Salas

Enlace de Youtube: <https://youtu.be/RqNnxhQkEvk>

Rúbrica de Evaluación

#	Ítem	Peso	Calif.
1	Corrección Diagrama ER de Chen	10	
2	Inventario de tablas resultantes del Modelo Lógico	10	

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Grupo 11

3	Tabla Comparativa de tipos de Datos	10	
4	Diccionario de Datos Físico (PostgreSQL)	80	
5	DDL - Scripts de CREACIÓN de Bases de Datos	120	
6	DDL - Scripts de MODIFICACIÓN de Bases de Datos	30	
7	Análisis de resultados (en equipo - 300 palabras mínimo)	20	
8	Reflexiones y Conclusiones individuales (300 palabras mínimo)	30	
9	Informe: Estructura, Contenido y Calidad de presentación	30	
10	Video de Sustentación	100	
11	Repositorio GIT	20	
12	Participación en Foro de Tarea	40	
NOTA = xx / 500 =		500	